Level 2
Hackathon – Fundamentals of Web Development

Student Details

Student Name	V. GOPIKA
Register Number	34625U50011
Department	BSC(AI)
ID	
College	M.M.E.S WOMEN'S ART AND SCIENCE COLLEGE
Date of Submission	28/10/2026

Git-Hub Link:

1. Problem Statement

Many students and individuals do not keep a proper record of their income and expenses. Money is spent on food, travel, bills and shopping without any tracking, so it is difficult to know where the money goes, how much is left, and which category consumes the most. Manual notes in books are easily lost, hard to search and cannot show summaries or charts.

A Personal Expense Tracker is needed to record income and expenses with amounts, dates, categories and descriptions, organize them, calculate the balance automatically, and present monthly summaries and charts so that users can monitor their spending and plan their finances better.

2. Objectives

The main objectives of the project are:

- To develop a web application that records income and expense transactions.
- To allow users to add, edit and delete transactions and manage expense categories.
- To provide search and filter options by description, category, transaction type and date range.
- To calculate total income, total expenses and current balance dynamically.
- To display monthly financial summaries and a simple expense chart by category or month.
- To store all data in the browser using Local Storage so that records are retained after refreshing the page.

Purpose and expected outcome: The application gives users a clear and easy way to analyze their financial activity. Whenever a record changes, the totals, summaries and chart update automatically, helping the user control spending.

3. Architecture

Application structure: The application is a single-page, client-side web application made of three files. **index.html** defines the layout (summary cards, transaction form, category manager, filters, transaction table, monthly summary and chart). **style.css** provides the design and responsive

layout. **script.js** contains all the logic: data handling, filtering, calculations, Local Storage and chart drawing. No server or database is required.

User navigation flow: The user opens the dashboard, adds a transaction using the form, manages categories, searches or filters the transaction list, edits or deletes records from the Actions column, and views the updated totals, monthly summary and expense chart on the same page.

Technologies used:

Technology	Role in the application
HTML5	Structures the page: forms, inputs, tables, buttons and the canvas element.
CSS3	Styles the interface; Grid and Flexbox layout, colors, cards and responsive design.
JavaScript (ES6)	Implements the logic: DOM updates, events, add/edit/delete, search/filter, calculations.
Local Storage	Stores transactions and categories in the browser as JSON data.
HTML5 Canvas	Draws the expense bar chart by category or by month.
VS Code / Web Browser	Used to write the code and to run and test the application.

Application flow diagram:

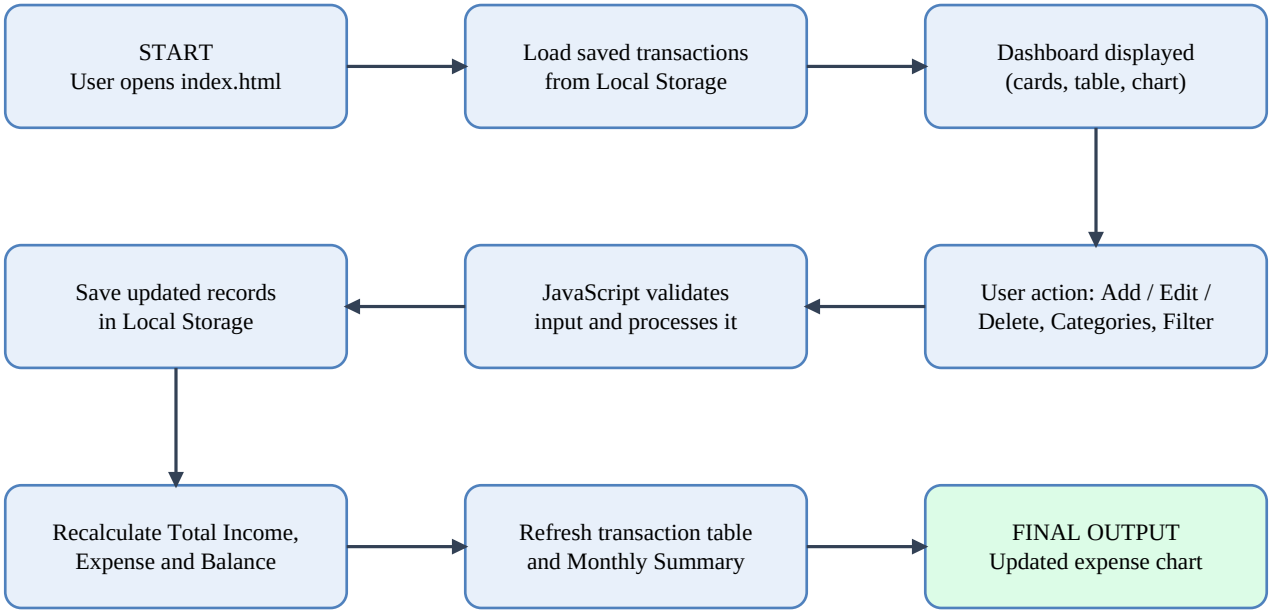


Figure A: Application flow from the initial page to the final output

4. Practical Implementation

The application was developed step by step. First, the page layout was designed in HTML with a summary section, a form panel and a transactions panel. Next, CSS was used to style the cards, tables, forms and chart area. Finally, JavaScript was written to handle form submission, store records in an array, save them to Local Storage, apply filters, calculate totals and draw the chart. A single function, **refresh()**, re-renders the totals, table, monthly summary and chart every time the

data changes, so the screen always matches the stored records.

4.1 Features Implemented

Feature	Purpose
Add / Edit / Delete transactions	Create new records, correct mistakes and remove unwanted entries.
Record income and expenses	Each record stores type, amount, date, category and description.
Manage categories	Add or remove categories; a category used in transactions cannot be deleted.
Search and filter	Find transactions by description, category and type.
Date / date range filter	Show only the transactions between the selected From and To dates.
Dynamic totals	Total income, total expenses and current balance are recalculated automatically.
Monthly financial summary	Shows income, expense and net amount for every month.
Local Storage	Keeps all data in the browser, even after the page is refreshed or closed.
Expense chart	Bar chart of expenses by category or by month.
Automatic updates	Summaries and chart update immediately whenever a record changes.

4.2 Source Code

The important sections of the source code and their functionality are given below.

HTML

HTML structures the application's content. The header shows the title; the summary section holds the income, expense and balance cards; the form panel is used to add or edit a transaction and manage categories; the transactions panel contains the search and filter controls and the table; the last section holds the monthly summary table and the canvas used for the chart.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Personal Expense Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="topbar">
    <h1>Personal Expense Tracker</h1>
    <p>Track your income and expenses easily</p>
  </header>

  <main class="container">

    <!-- Summary cards -->
    <section class="summary">
      <div class="card income">
        <span>Total Income</span>
        <strong id="totalIncome">₹0.00</strong>
      </div>
      <div class="card expense">
        <span>Total Expenses</span>
        <strong id="totalExpense">₹0.00</strong>
      </div>
      <div class="card balance">
        <span>Current Balance</span>
        <strong id="balance">₹0.00</strong>
      </div>
    </section>

    <div class="grid">
      <!-- Add / Edit transaction form -->
      <section class="panel">
        <h2 id="formTitle">Add Transaction</h2>
        <form id="txForm">
          <input type="hidden" id="txId">

          <label for="type">Type</label>
          <select id="type">
            <option value="income">Income</option>
            <option value="expense" selected>Expense</option>
          </select>

          <label for="amount">Amount (₹)</label>
          <input type="number" id="amount" min="0" step="0.01" placeholder="e.g. 500" required>

          <label for="date">Date</label>
          <input type="date" id="date" required>

          <label for="category">Category</label>
          <select id="category" required></select>

          <label for="description">Description</label>
          <input type="text" id="description" placeholder="e.g. Lunch at canteen" required>

          <div class="btn-row">
            <button type="submit" class="btn primary" id="saveBtn">Add Transaction</button>
            <button type="button" class="btn ghost" id="cancelBtn" hidden>Cancel</button>
          </div>
        </form>

        <hr>

        <h2>Manage Categories</h2>
        <div class="cat-add">
          <input type="text" id="newCategory" placeholder="New category name">
          <button type="button" class="btn primary" id="addCatBtn">Add</button>
        </div>
        <ul id="catList" class="chips"></ul>
      </section>
      <!-- Transactions + filters -->
```

```
      <section class="panel wide">
        <h2>Transactions</h2>
        <div class="filters">
          <input type="text" id="search" placeholder="Search description...">
          <select id="filterCategory"></select>
          <select id="filterType">
            <option value="all">All Types</option>
            <option value="income">Income</option>
            <option value="expense">Expense</option>
          </select>
          <input type="date" id="fromDate" title="From date">
          <input type="date" id="toDate" title="To date">
          <button type="button" class="btn ghost" id="clearFilters">Clear</button>
        </div>

        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th>Date</th><th>Description</th><th>Category</th>
                <th>Type</th><th class="r">Amount</th><th class="r">Actions</th>
              </tr>
            </thead>
            <tbody id="txBody"></tbody>
          </table>
          <p id="emptyMsg" class="empty" hidden>No transactions found.</p>
        </div>
      </section>
    </div>

    <!-- Monthly summary + chart -->
    <div class="grid two">
      <section class="panel">
        <h2>Monthly Summary</h2>
        <table class="mini">
          <thead>
            <tr><th>Month</th><th class="r">Income</th><th class="r">Expense</th><th class="r">Net</th></tr>
          </thead>
          <tbody id="monthBody"></tbody>
        </table>
      </section>

      <section class="panel">
        <div class="chart-head">
          <h2>Expense Chart</h2>
          <select id="chartMode">
            <option value="category">By Category</option>
            <option value="month">By Month</option>
          </select>
        </div>
        <canvas id="chart" width="520" height="300"></canvas>
      </section>
    </div>
  </main>

  <footer class="footer">Personal Expense Tracker &bull; Data is saved in your browser (Local Storage)</footer>

  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS styles the application. Color variables are defined in **:root**, CSS Grid arranges the summary cards and panels, tables and forms get a clean card-style look, income and expense amounts use green and red colors, and a media query makes the layout responsive for small screens.

```
* { box-sizing: border-box; margin: 0; padding: 0; }

:root {
  --primary: #2563eb;
  --income: #16a34a;
  --expense: #dc2626;
  --bg: #f1f5f9;
  --text: #1e293b;
  --muted: #64748b;
  --border: #e2e8f0;
}

body {
  font-family: "Segoe UI", Arial, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.4;
}
.topbar {
```

```
  background: linear-gradient(135deg, #1d4ed8, #2563eb);
  color: #fff;
  padding: 18px 32px;
}
.topbar h1 { font-size: 24px; }
.topbar p  { font-size: 13px; opacity: .85; margin-top: 2px; }

.container { max-width: 1280px; margin: 0 auto; padding: 20px; }

/* Summary cards */
.summary { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; margin-bottom: 18px; }
.card {
  background: #fff; border-radius: 10px; padding: 16px 20px;
  border-left: 5px solid var(--primary);
  box-shadow: 0 1px 4px rgba(0,0,0,.08);
  display: flex; flex-direction: column; gap: 4px;
}
.card span   { color: var(--muted); font-size: 13px; }
.card strong { font-size: 26px; }
.card.income  { border-color: var(--income); }
.card.income strong  { color: var(--income); }
.card.expense { border-color: var(--expense); }
.card.expense strong { color: var(--expense); }

/* Layout */
.grid { display: grid; grid-template-columns: 340px 1fr; gap: 18px; margin-bottom: 18px; align-items: start; }
.grid.two { grid-template-columns: 1fr 1fr; }
.panel {
  background: #fff; border-radius: 10px; padding: 18px;
  box-shadow: 0 1px 4px rgba(0,0,0,.08);
}
.panel h2 { font-size: 16px; margin-bottom: 12px; }
hr { border: none; border-top: 1px solid var(--border); margin: 18px 0; }

/* Form */
label { display: block; font-size: 12px; color: var(--muted); margin: 10px 0 4px; }
input, select {
  width: 100%; padding: 8px 10px; font-size: 14px;
  border: 1px solid var(--border); border-radius: 6px; background: #fff;
}
input:focus, select:focus { outline: 2px solid #bfdbfe; border-color: var(--primary); }

.btn { padding: 8px 14px; font-size: 14px; border: none; border-radius: 6px; cursor: pointer; }
.btn.primary { background: var(--primary); color: #fff; }
.btn.primary:hover { background: #1d4ed8; }
.btn.ghost { background: #e2e8f0; color: var(--text); }
.btn-row { display: flex; gap: 8px; margin-top: 14px; }
.btn-row .btn { flex: 1; }

/* Categories */
.cat-add { display: flex; gap: 8px; }
.cat-add input { flex: 1; }
.chips { list-style: none; display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
.chips li {
  background: #eff6ff; color: #1d4ed8; border-radius: 999px;
  padding: 4px 6px 4px 12px; font-size: 12px; display: flex; align-items: center; gap: 6px;
}
.chips button {
  border: none; background: #dbeafe; color: #1d4ed8; width: 18px; height: 18px;
  border-radius: 50%; cursor: pointer; font-size: 12px; line-height: 1;
}

/* Filters */
.filters { display: grid; grid-template-columns: 1.4fr 1fr 1fr 1fr 1fr auto; gap: 8px; margin-bottom: 12px; }

/* Tables */
.table-wrap { max-height: 360px; overflow: auto; border: 1px solid var(--border); border-radius: 8px; }
table { width: 100%; border-collapse: collapse; font-size: 13px; }
th, td { padding: 9px 10px; text-align: left; border-bottom: 1px solid var(--border); }
th { background: #f8fafc; position: sticky; top: 0; color: var(--muted); font-weight: 600; }
.r { text-align: right; }
td.inc { color: var(--income); font-weight: 600; text-align: right; }
td.exp { color: var(--expense); font-weight: 600; text-align: right; }
.badge { padding: 2px 8px; border-radius: 999px; font-size: 11px; font-weight: 600; }
.badge.income  { background: #dcfce7; color: #166534; }
.badge.expense { background: #fee2e2; color: #991b1b; }
.act { border: none; background: none; cursor: pointer; font-size: 12px; padding: 3px 6px; border-radius: 4px; }
.act.edit { color: var(--primary); }
.act.del  { color: var(--expense); }
.act:hover { background: #f1f5f9; }
.empty { text-align: center; padding: 20px; color: var(--muted); }
.mini { border: 1px solid var(--border); border-radius: 8px; overflow: hidden; }

/* Chart */
.chart-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; }
.chart-head select { width: auto; }
canvas { width: 100%; height: auto; }

.footer { text-align: center; color: var(--muted); font-size: 12px; padding: 14px; }
@media (max-width: 900px) {
```

```
  .summary, .grid, .grid.two { grid-template-columns: 1fr; }
  .filters { grid-template-columns: 1fr 1fr; }
}
```

JavaScript

JavaScript implements the application logic. It loads and saves data in Local Storage, adds, edits and deletes transactions, manages categories, filters records, calculates totals and monthly summaries, draws the chart on the canvas and calls **refresh()** to keep the screen updated.

```
// ---------- Storage keys & state ----------
const TX_KEY  = "pet_transactions";
const CAT_KEY = "pet_categories";

let transactions = JSON.parse(localStorage.getItem(TX_KEY)  || "[]");
let categories   = JSON.parse(localStorage.getItem(CAT_KEY) || '["Food","Travel","Shopping","Bills","Salary","Education"]');

// ---------- DOM references ----------
const $ = (id) => document.getElementById(id);
const form = $("txForm");

// ---------- Helpers ----------
const money = (n) => "₹" + n.toLocaleString("en-IN", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
const save  = () => {
  localStorage.setItem(TX_KEY, JSON.stringify(transactions));
  localStorage.setItem(CAT_KEY, JSON.stringify(categories));
};

// ---------- Categories ----------
function renderCategories() {
  const options = categories.map((c) => `<option value="${c}">${c}</option>`).join("");
  const current = $("category").value;
  $("category").innerHTML = options;
  if (categories.includes(current)) $("category").value = current;

  const fcur = $("filterCategory").value || "all";
  $("filterCategory").innerHTML = '<option value="all">All Categories</option>' + options;
  $("filterCategory").value = categories.includes(fcur) ? fcur : "all";

  $("catList").innerHTML = categories
    .map((c) => `<li>${c}<button title="Delete" onclick="deleteCategory('${c}')">×</button></li>`)
    .join("");
}

$("addCatBtn").addEventListener("click", () => {
  const name = $("newCategory").value.trim();
  if (!name) return;
  if (categories.some((c) => c.toLowerCase() === name.toLowerCase())) {
    alert("Category already exists.");
    return;
  }
  categories.push(name);
  $("newCategory").value = "";
  save();
  renderCategories();
});

function deleteCategory(name) {
  if (transactions.some((t) => t.category === name)) {
    alert("This category is used in transactions and cannot be deleted.");
    return;
  }
  categories = categories.filter((c) => c !== name);
  save();
  renderCategories();
}

// ---------- Add / Edit / Delete transactions ----------
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const amount = parseFloat($("amount").value);
  if (!(amount > 0)) { alert("Enter a valid amount."); return; }

  const data = {
    id: $("txId").value || String(Date.now()),
    type: $("type").value,
    amount,
    date: $("date").value,
    category: $("category").value,
    description: $("description").value.trim(),
  };

  const idx = transactions.findIndex((t) => t.id === data.id);
  if (idx >= 0) transactions[idx] = data; else transactions.push(data);

  save();
```

```
  resetForm();
  refresh();
});

function editTransaction(id) {
  const t = transactions.find((x) => x.id === id);
  if (!t) return;
  $("txId").value = t.id;
  $("type").value = t.type;
  $("amount").value = t.amount;
  $("date").value = t.date;
  $("category").value = t.category;
  $("description").value = t.description;
  $("formTitle").textContent = "Edit Transaction";
  $("saveBtn").textContent = "Update Transaction";
  $("cancelBtn").hidden = false;
  window.scrollTo({ top: 0, behavior: "smooth" });
}

function deleteTransaction(id) {
  if (!confirm("Delete this transaction?")) return;
  transactions = transactions.filter((t) => t.id !== id);
  save();
  refresh();
}

function resetForm() {
  form.reset();
  $("txId").value = "";
  $("date").value = new Date().toISOString().slice(0, 10);
  $("formTitle").textContent = "Add Transaction";
  $("saveBtn").textContent = "Add Transaction";
  $("cancelBtn").hidden = true;
}
$("cancelBtn").addEventListener("click", resetForm);

// ---------- Search & filter ----------
function getFiltered() {
  const q    = $("search").value.trim().toLowerCase();
  const cat  = $("filterCategory").value;
  const type = $("filterType").value;
  const from = $("fromDate").value;
  const to   = $("toDate").value;

  return transactions
    .filter((t) =>
      (!q || t.description.toLowerCase().includes(q) || t.category.toLowerCase().includes(q)) &&
      (cat === "all" || t.category === cat) &&
      (type === "all" || t.type === type) &&
      (!from || t.date >= from) &&
      (!to || t.date <= to)
    )
    .sort((a, b) => b.date.localeCompare(a.date));
}

["search", "filterCategory", "filterType", "fromDate", "toDate"].forEach((id) =>
  $(id).addEventListener("input", renderTable)
);
$("clearFilters").addEventListener("click", () => {
  $("search").value = ""; $("filterCategory").value = "all"; $("filterType").value = "all";
  $("fromDate").value = ""; $("toDate").value = "";
  renderTable();
});

// ---------- Render table ----------
function renderTable() {
  const list = getFiltered();
  $("txBody").innerHTML = list.map((t) => `
    <tr>
      <td>${t.date}</td>
      <td>${t.description}</td>
      <td>${t.category}</td>
      <td><span class="badge ${t.type}">${t.type === "income" ? "Income" : "Expense"}</span></td>
      <td class="${t.type === "income" ? "inc" : "exp"}">${t.type === "income" ? "+" : "-"}${money(t.amount)}</td>
      <td class="r">
        <button class="act edit" onclick="editTransaction('${t.id}')">Edit</button>
        <button class="act del" onclick="deleteTransaction('${t.id}')">Delete</button>
      </td>
    </tr>`).join("");
  $("emptyMsg").hidden = list.length > 0;
}

// ---------- Totals (calculated dynamically) ----------
function renderTotals() {
  const inc = transactions.filter((t) => t.type === "income").reduce((s, t) => s + t.amount, 0);
  const exp = transactions.filter((t) => t.type === "expense").reduce((s, t) => s + t.amount, 0);
  $("totalIncome").textContent  = money(inc);
  $("totalExpense").textContent = money(exp);
  $("balance").textContent      = money(inc - exp);
  $("balance").style.color      = inc - exp >= 0 ? "#16a34a" : "#dc2626";
```

```
}

// ---------- Monthly summary ----------
function monthlyData() {
  const map = {};
  transactions.forEach((t) => {
    const m = t.date.slice(0, 7);
    map[m] = map[m] || { income: 0, expense: 0 };
    map[m][t.type] += t.amount;
  });
  return Object.keys(map).sort().map((m) => ({ month: m, ...map[m] }));
}

function monthLabel(ym) {
  const [y, m] = ym.split("-");
  return new Date(y, m - 1).toLocaleString("en-US", { month: "short", year: "numeric" });
}

function renderMonthly() {
  const rows = monthlyData();
  $("monthBody").innerHTML = rows.length
    ? rows.map((r) => `
      <tr>
        <td>${monthLabel(r.month)}</td>
        <td class="inc">${money(r.income)}</td>
        <td class="exp">${money(r.expense)}</td>
        <td class="r"><strong>${money(r.income - r.expense)}</strong></td>
      </tr>`).join("")
    : '<tr><td colspan="4" class="empty">No data yet.</td></tr>';
}

// ---------- Chart (Canvas bar chart) ----------
function chartData() {
  if ($("chartMode").value === "month") {
    return monthlyData().map((r) => ({ label: monthLabel(r.month), value: r.expense }));
  }
  const map = {};
  transactions.filter((t) => t.type === "expense").forEach((t) => {
    map[t.category] = (map[t.category] || 0) + t.amount;
  });
  return Object.entries(map).map(([label, value]) => ({ label, value }));
}

function renderChart() {
  const canvas = $("chart");
  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);

  const data = chartData();
  if (!data.length) {
    ctx.fillStyle = "#64748b"; ctx.font = "14px Segoe UI, Arial"; ctx.textAlign = "center";
    ctx.fillText("No expense data to display", W / 2, H / 2);
    return;
  }

  const colors = ["#2563eb", "#f59e0b", "#16a34a", "#dc2626", "#8b5cf6", "#0891b2", "#db2777"];
  const pad = { l: 55, r: 15, t: 20, b: 45 };
  const max = Math.max(...data.map((d) => d.value)) * 1.15;
  const chartH = H - pad.t - pad.b;
  const chartW = W - pad.l - pad.r;
  const bw = Math.min(60, chartW / data.length * 0.6);

  // grid lines + y labels
  ctx.font = "11px Segoe UI, Arial"; ctx.textAlign = "right"; ctx.strokeStyle = "#e2e8f0";
  for (let i = 0; i <= 4; i++) {
    const y = pad.t + chartH - (chartH * i) / 4;
    ctx.beginPath(); ctx.moveTo(pad.l, y); ctx.lineTo(W - pad.r, y); ctx.stroke();
    ctx.fillStyle = "#64748b";
    ctx.fillText("₹" + Math.round((max * i) / 4), pad.l - 6, y + 4);
  }

  // bars
  data.forEach((d, i) => {
    const slot = chartW / data.length;
    const x = pad.l + slot * i + (slot - bw) / 2;
    const h = (d.value / max) * chartH;
    const y = pad.t + chartH - h;
    ctx.fillStyle = colors[i % colors.length];
    ctx.fillRect(x, y, bw, h);

    ctx.fillStyle = "#1e293b"; ctx.textAlign = "center";
    ctx.fillText("₹" + Math.round(d.value), x + bw / 2, y - 5);
    ctx.fillStyle = "#64748b";
    ctx.fillText(d.label, x + bw / 2, H - pad.b + 16);
  });
}
$("chartMode").addEventListener("change", renderChart);

// ---------- Refresh everything when records change ----------
```

```
function refresh() {
  renderTotals();
  renderTable();
  renderMonthly();
  renderChart();
}

// ---------- Init ----------
renderCategories();
resetForm();
refresh();
```

4.3 Project Execution

How to run the application:

- Create a project folder and save the three files as **index.html**, **style.css** and **script.js** in the same folder.
- Open the folder in VS Code and run **index.html** with Live Server, or simply double-click **index.html** to open it in a web browser.
- The Personal Expense Tracker dashboard opens. No installation or internet connection is needed.

Working process (input to output):

- The user selects the type, enters amount, date, category and description, and clicks **Add Transaction**.
- JavaScript validates the input, creates a record and stores it in the transactions array and in Local Storage.
- The totals, transaction table, monthly summary and expense chart are updated immediately by **refresh()**.
- The user can search or filter records, edit or delete any record, and add new categories; each change updates the output again.

5. Output Screenshots

The following screenshots show the interface, the implemented features and the final results of the working application.

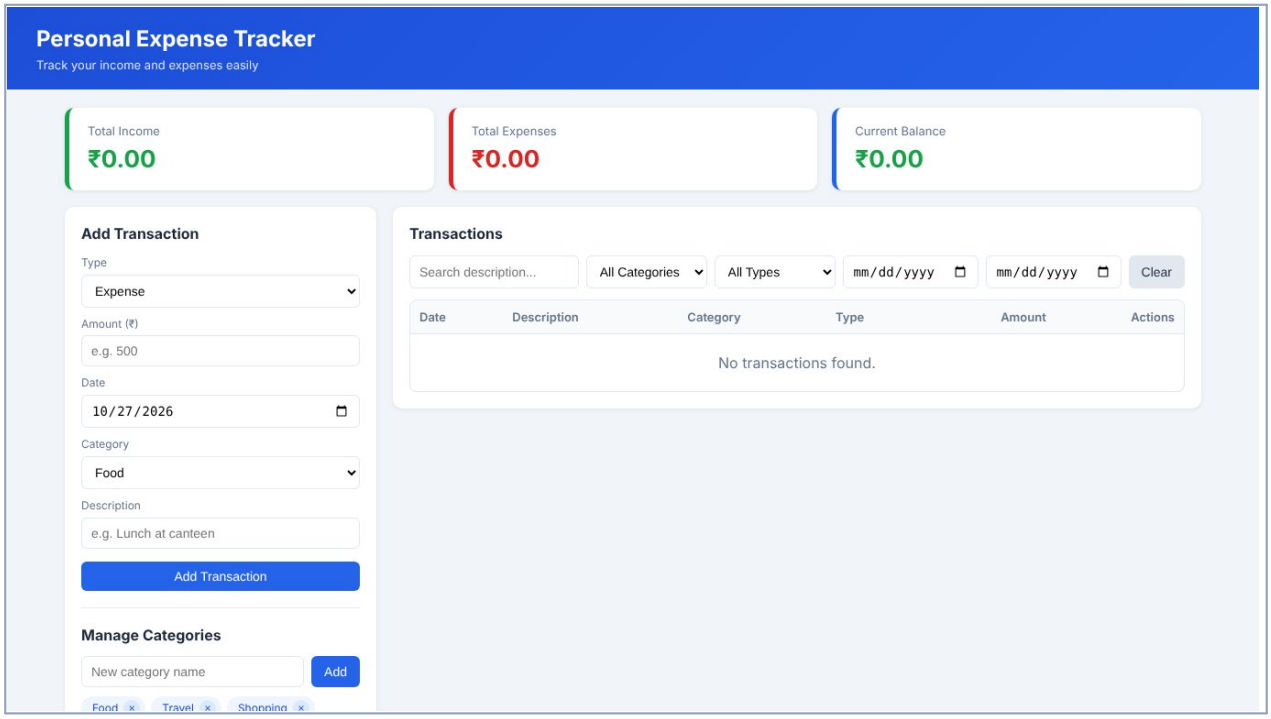


Figure 1: Home page of the Personal Expense Tracker (initial state)

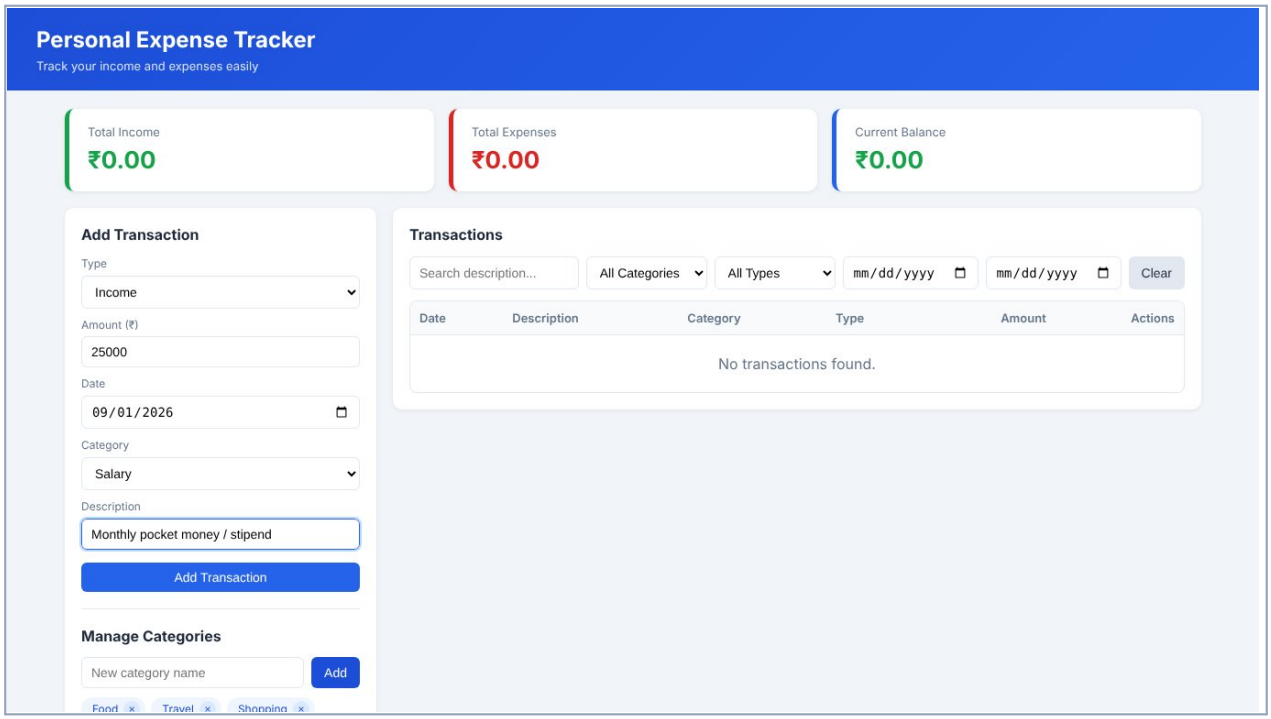


Figure 2: Add Transaction form filled with income details and a new category added

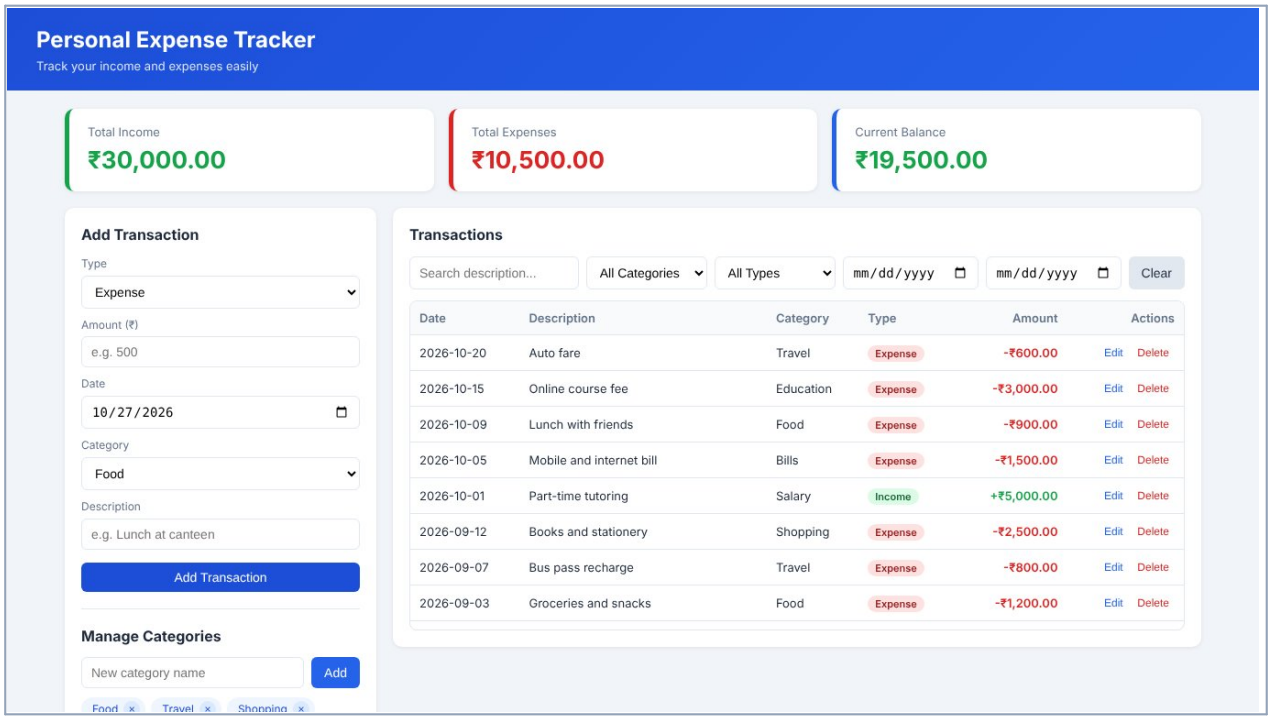


Figure 3: Dashboard showing transactions with total income, total expenses and current balance

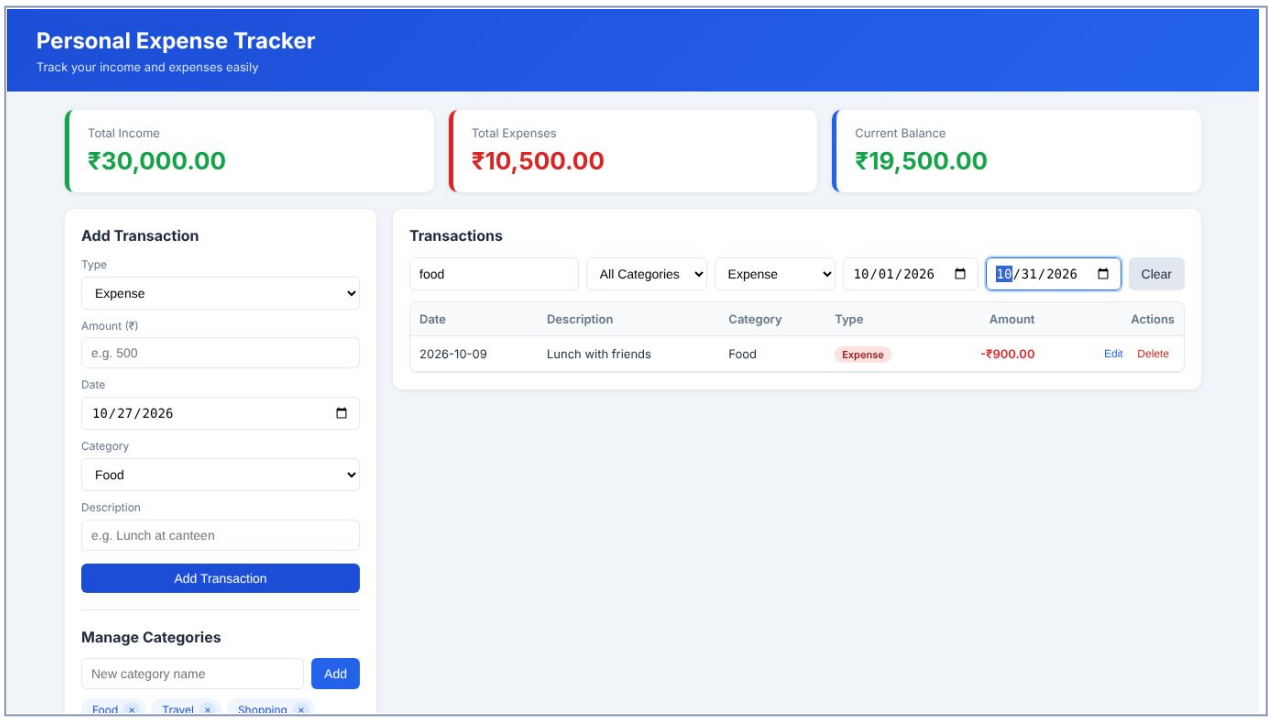


Figure 4: Search and filter by description, transaction type and date range

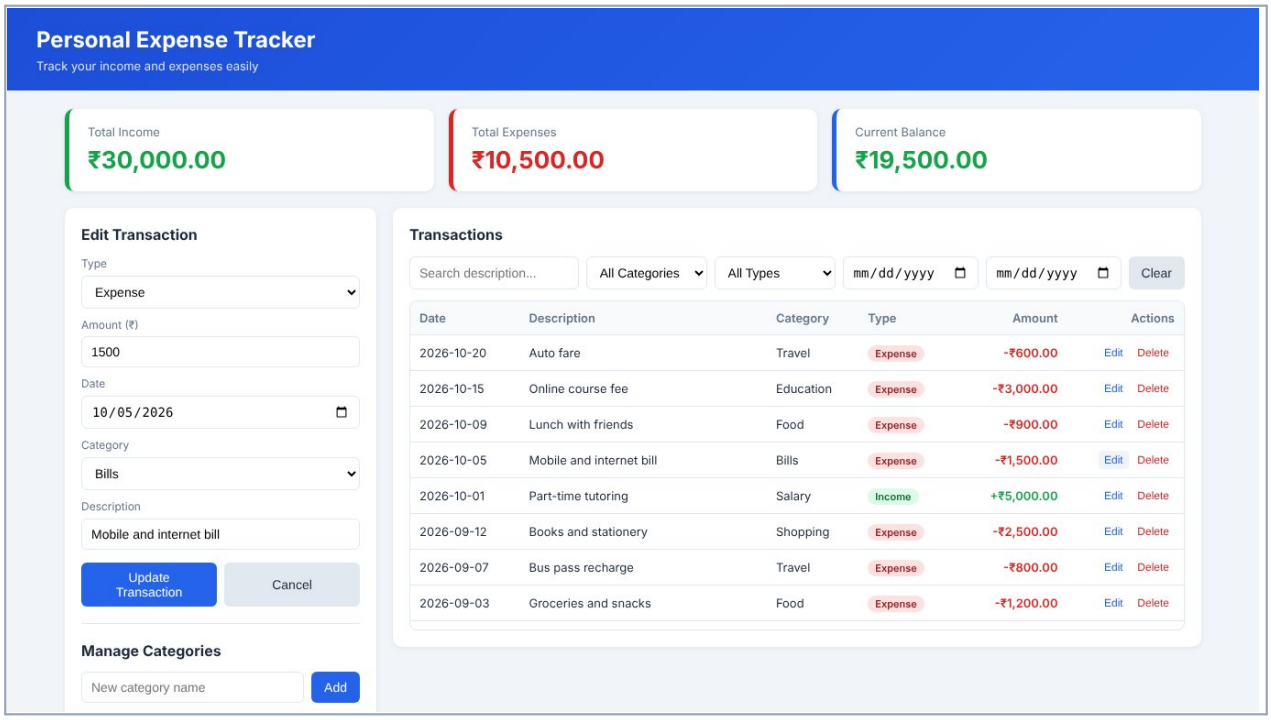


Figure 5: Editing an existing transaction (Update Transaction mode)

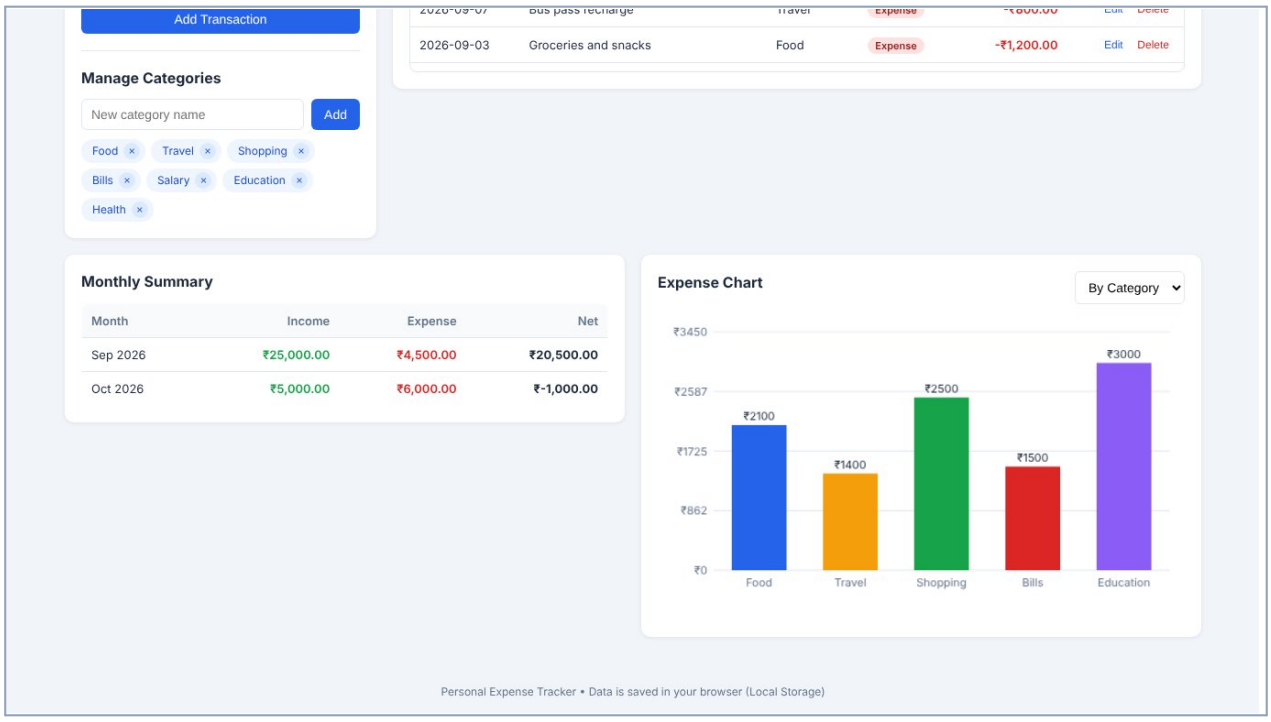


Figure 6: Monthly summary and expense chart by category

6. Technical Explanation

The application is built with core web technologies. **HTML** provides semantic structure with form controls, tables and a canvas. **CSS** uses variables, Flexbox, Grid and media queries for styling and responsiveness. **JavaScript** controls the behavior of the application.

- **DOM manipulation and events:** *getElementById*, *addEventListener* and template literals are used to read inputs and render table rows, category chips and summaries.

- **Array methods:** *filter()* for search and date filtering, *reduce()* for totals, *map()* for rendering and *sort()* to show the latest transactions first.
- **CRUD operations:** a unique id (timestamp) identifies each record so that it can be edited or deleted; the same form is reused for add and edit.
- **Local Storage with JSON:** *JSON.stringify()* saves the arrays and *JSON.parse()* loads them when the page opens.
- **Dynamic calculation:** income, expenses and balance are computed from the data every time, never stored separately, so they are always correct.
- **Monthly grouping:** dates in YYYY-MM-DD format are grouped by their first seven characters (year-month) using an object as a map.
- **Canvas chart:** bar heights are scaled using the maximum value; grid lines, labels and bars are drawn with the Canvas 2D API.
- **Validation:** required fields and a positive amount check prevent invalid data; a category in use cannot be deleted.

7. Conclusion

The Personal Expense Tracker was successfully developed using HTML, CSS and JavaScript. The application allows users to add, edit and delete transactions, manage categories, search and filter records, view total income, total expenses and balance, and analyze spending through a monthly summary and an expense chart. All data is saved in Local Storage, and every change updates the summaries and chart automatically.

Through this project, I learned DOM manipulation, event handling, array methods, JSON and Local Storage, form validation, responsive layout design and drawing charts using the HTML5 Canvas. The project helped me understand how a complete front-end application is planned, developed and tested.

8. Future Enhancements

- Add user login and registration with secure password handling.
- Store data in a cloud database so that records can be accessed from any device.
- Add budget limits with alerts when spending crosses the limit.
- Export reports to PDF or Excel and import data from files.
- Add more chart types such as pie and line charts, and yearly reports.
- Add recurring transactions, dark mode and a mobile app version for a better user experience.